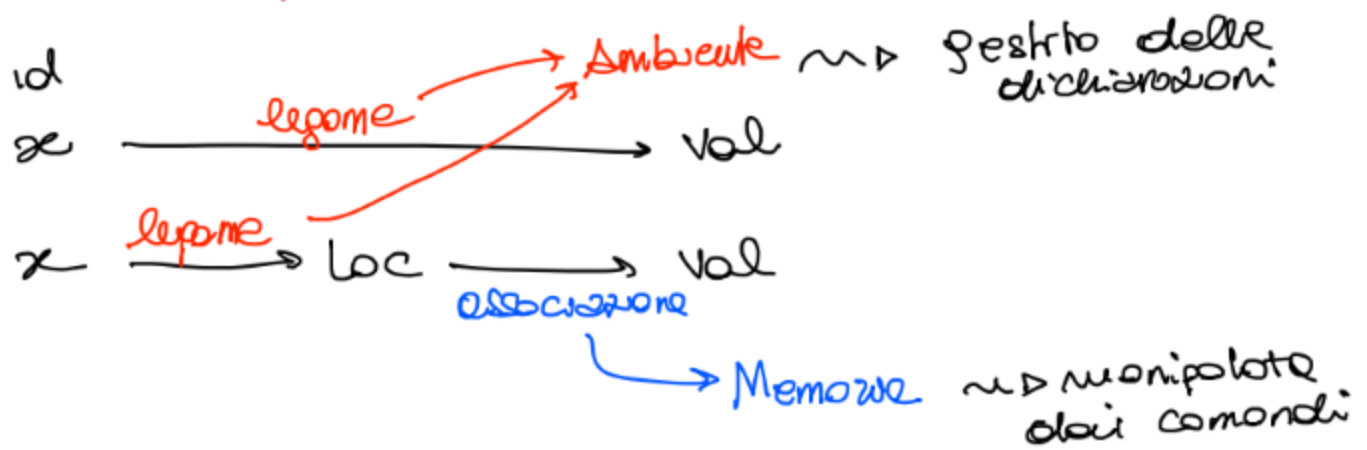


AMBIENTI VS MEMORIE



⇒ "interpretazione" tra memoria e ambiente

come creiamo l'ambiente in cui
vogliamo eseguire del codice

Blocco ⇒ comandi

↳ Porzione di programma (potenzialmente isolata da
altri pezzi di codice)

Blocchi:

→ Permettono di rendere modulare il codice
e quindi di renderlo più chiaro

→ Permettono la gestione locale di identificatori

→ Permettono di gestire l'allocazione della memoria

→ Permettono la ricorsione

maggiormente legati ai blocchi come procedure

D; C

↳ blocco inline (senza nome)

Specifica le dichiarazioni D
che definiscono l'ambiente in
cui eseguire la porzione di
codice C

{ blocco } ← alcuni linguaggi possono richiedere delimitatori del blocco

In IMP $C \rightarrow \dots | \overbrace{D; C}^{\text{comando}}$

⇒ e' possibile inserire dichiarazioni in qualunque punto del programma

⇒ con questa sintassi (e con la semantica che specificheremo) una dichiarazione e' visibile da dove viene inserito fino alla fine del programma

$D \xrightarrow{\text{visibilit\`a}}$

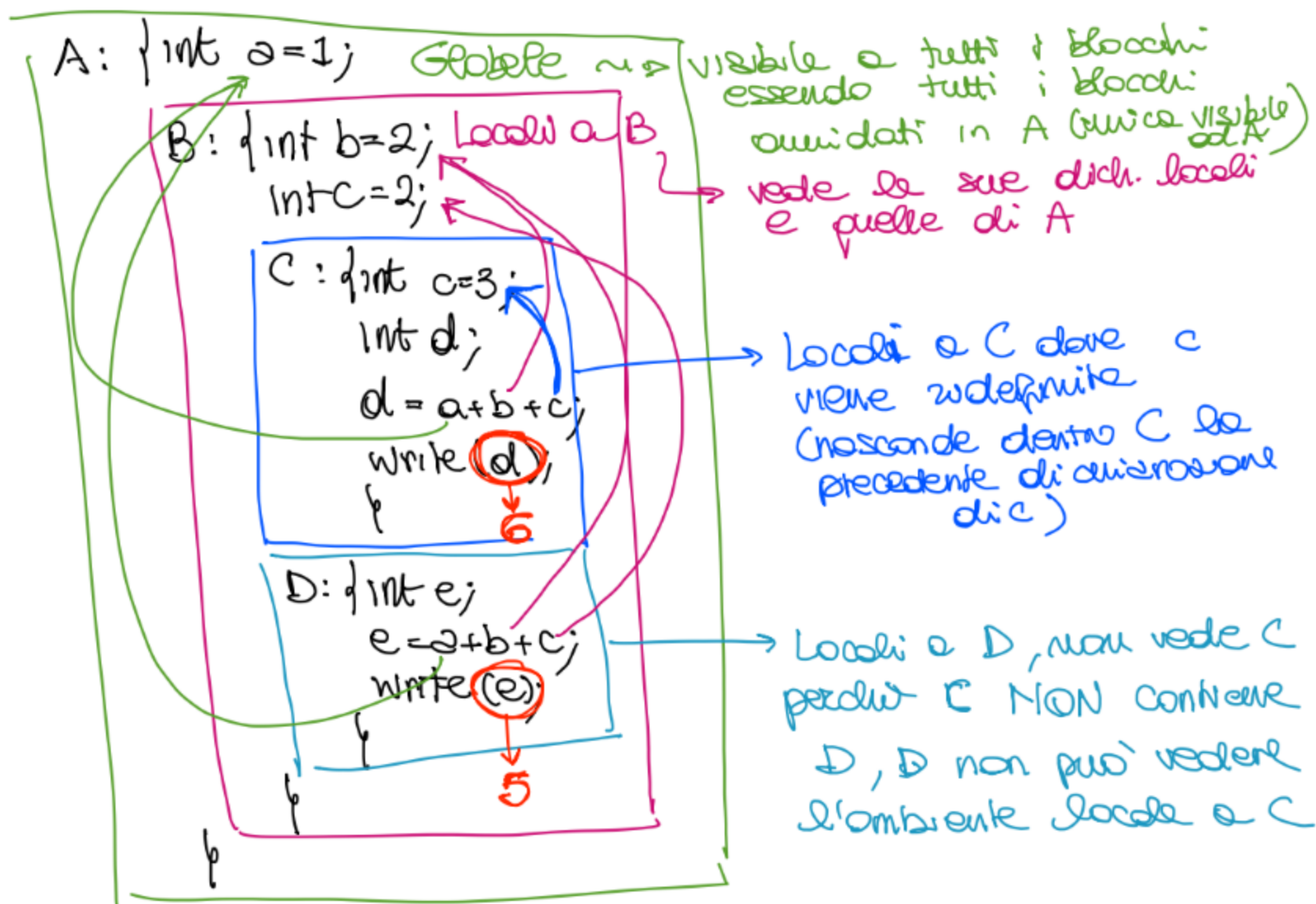
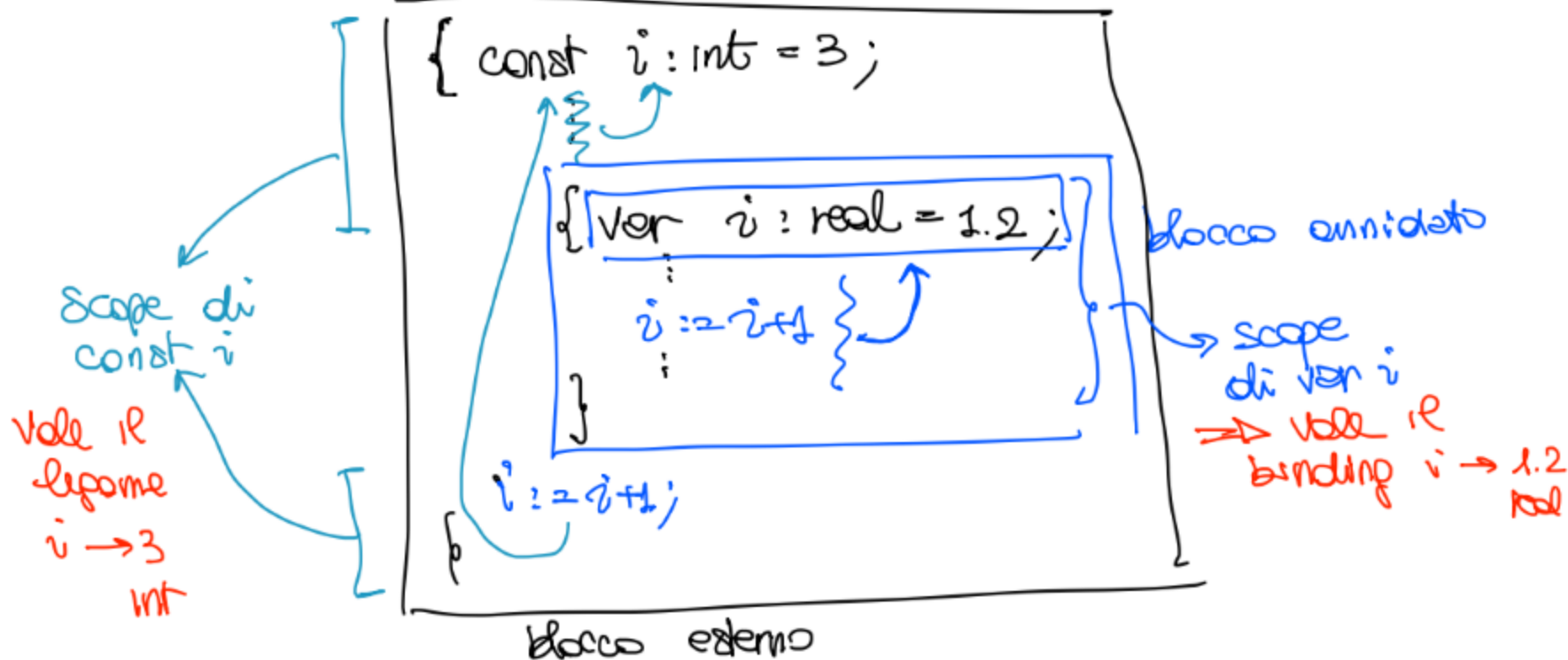
Blocchi annidati



⇒ I blocchi possono solo essere contenuti internamente in altri blocco ⇒ blocchi annidati

⇒ Questa regola di sovrapposizione tra blocchi permette di definire la regola di visibilit\`a delle dichiarazioni

Regola di visibilit\`a: Una dichiarazione locale ad un blocco e' visibile nel blocco e in tutti i blocchi annidati, a meno che in tali blocchi non intervenga una dichiarazione dello stesso nome.



SCOPE $\Rightarrow$ porzione di codice in cui tutte le occorrenze di `x` fanno riferimento alla dichiarazione di `x`

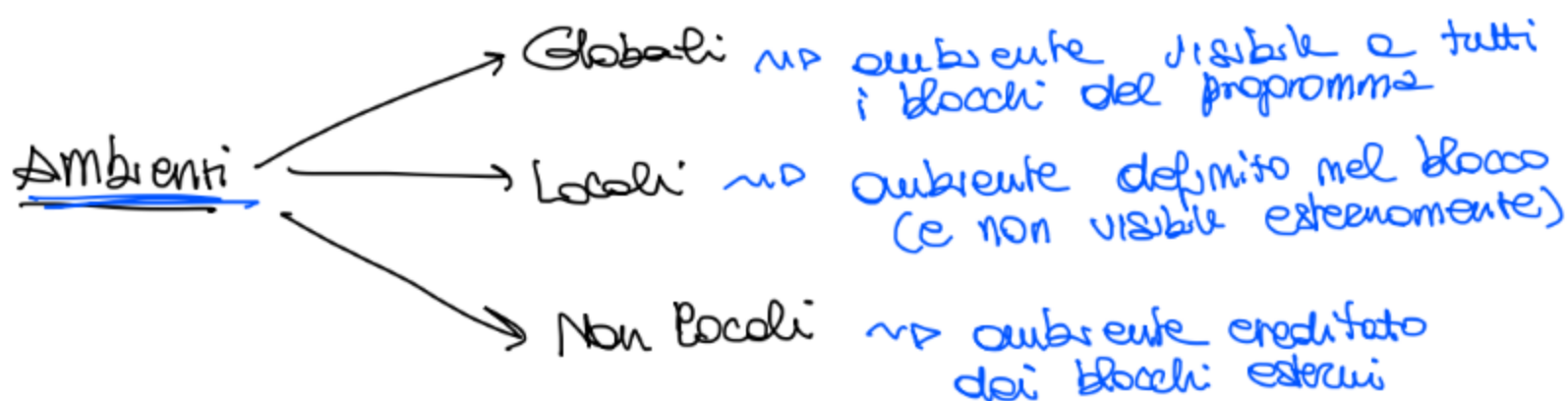
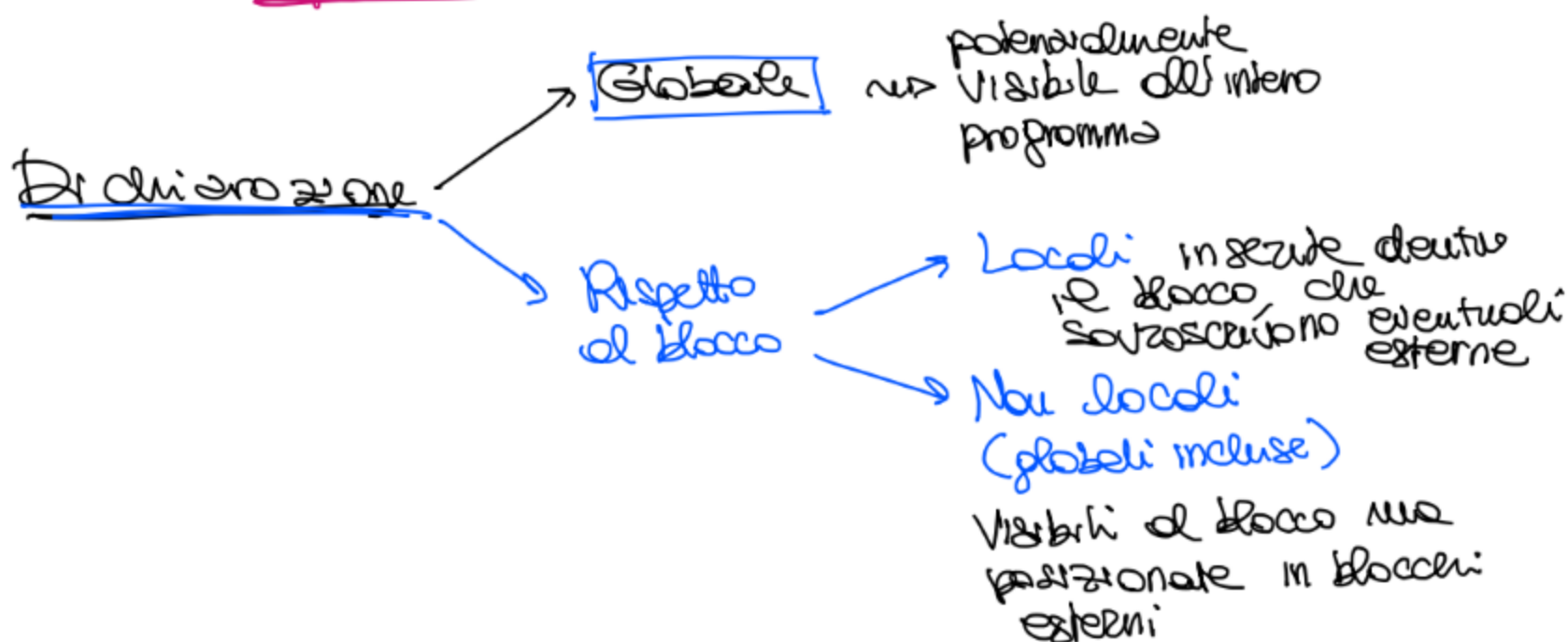
2 $\Rightarrow$ devono usare il legame definito dalla dichiarazione

Var x: int = 3



regole di visibilità definisce lo scope ovvero la porzione di codice in cui le occorrenze della variabile dichiarata fanno riferimento alla dichiarazione

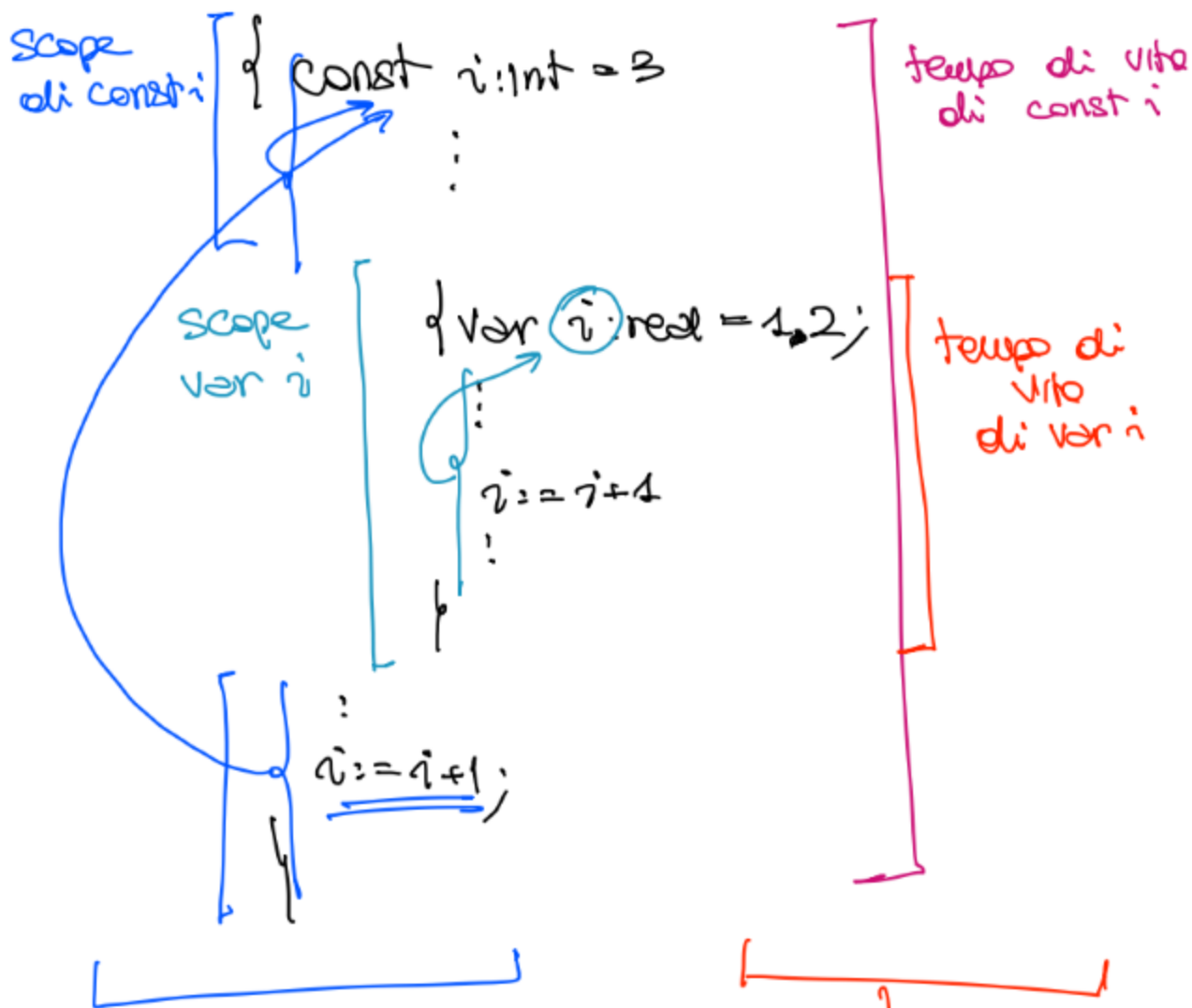
→ concetto statico che lega un identificatore all'ambiente di riferimento. a tempo di compilazione in quanto (senza l'uso di procedure) eseguimo il codice dove viene definito



SCOPING → concetto che riguarda porzioni di codice
⇒ determina e quali legami (ambiente) fa riferimento ogni occorrenza
come di riferimento nel codice
"x"

TEMPO DI VITA → ogni variabile ha un tempo di vita
ovvero il tempo di esecuzione in cui
l'associazione della locazione (memoria)
è valida.

[Allocazione in memoria
Dedlocazione



↓
concetto statico nel senso
che è completamente
determinabile da codice
fisso e regole (visibilità, ...)

↓
concetto dinamico
nel senso che dipende
dall'esecuzione
del codice

$\{ D_0$ dich. $\rightarrow P_0$
 while ...
 :

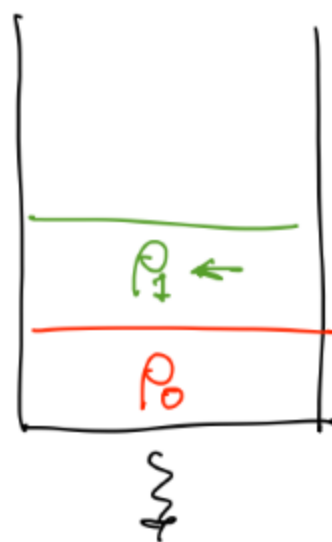
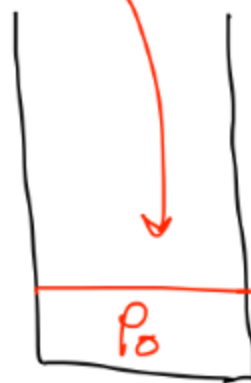
$\{ D_1$ dich. $\rightarrow P_1$
 C_{11} comandi

$\{ D_2$ dich. $\rightarrow P_2$
 C_{12} comandi

$C_{2,3}$ comandi

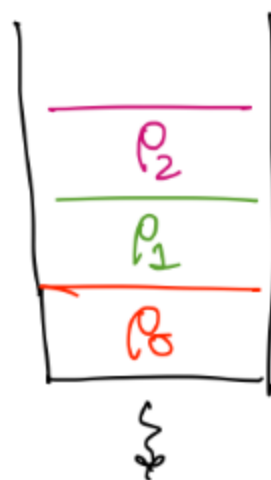
:

{



Entrata in D_1

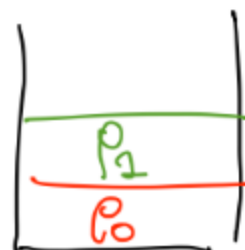
P_2 nello stack = tempo
 di vita
 delle vor
 nel suo
 ambiente



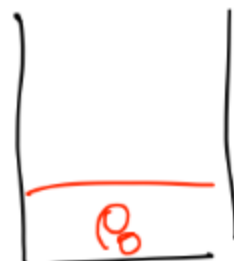
Entrata in D_2

tempo di
 vita vor
 in P_1

uscita da
 D_2



uscita
 da D_1



termina il ciclo

Allocazioni di p_1 e p_2 successive nella stack durante l'esecuzione del while sono di fatto subventi nuovi, diversi.

2 → sono stati distrutti e rallocati ad ogni iterazione

Blocchi in IMP

com : $C \rightarrow x := E \mid \text{skip} \mid \text{while } E \text{ do } C$
 $\mid \text{if } E \text{ then } C \text{ else } C$
 $\boxed{D; C}$ blocco

Semantica statica → determina se il comando è ben formato

$$\frac{\Delta \vdash d : \Delta_0 \quad \Delta[\Delta_0] \vdash C}{\Delta \vdash d; C}$$

$$FI(d; C) = FI(d) \cup (FI(C) \setminus DI(d))$$

posizione
libero

$$DI(d; C) = DI(d) \cup DI(C)$$

Semantica dinamica

$$\frac{\rho \vdash \langle d, \sigma \rangle \rightarrow^* \langle p_0, \sigma' \rangle}{\rho \vdash \langle d; C, \sigma \rangle \rightarrow \langle p_0; C, \sigma' \rangle}$$

$$\frac{\longrightarrow p[p_0] \vdash \langle c, \sigma \rangle \longrightarrow^* \sigma'}{p \vdash \langle p_0; c, \sigma \rangle \longrightarrow \sigma'}$$

Esempio

$\underbrace{\text{var } x: \text{int} = 3;}_d \underbrace{x := x + 1}_c$

$$\frac{\vdash d : \Delta \quad \Delta \vdash c}{\vdash d; c} \text{ Repole } \textcircled{*}$$

$$\frac{\vdash 3 : \text{int}}{\vdash \text{var } x: \text{int} = 3 : [x \mapsto \text{intloc}] \equiv \Delta}$$

Riscriviamo la regola $\textcircled{*}$

$$\frac{\vdash d : [x \mapsto \text{intloc}] \quad [x \mapsto \text{intloc}] \vdash c}{\vdash d; c}$$

$$\frac{[x \mapsto \text{intloc}] \vdash x + 1 : ?}{[x \mapsto \text{intloc}] \vdash x := x + 1} \quad \Delta(x) = \text{intloc}$$

$$\frac{[x \mapsto \text{intloc}] \vdash x : \text{int} \quad [x \mapsto \text{intloc}] \vdash 1 : \text{int}}{[x \mapsto \text{intloc}] \vdash x + 1 : \text{int}} \quad \Delta(x) = \text{int}$$

$\Rightarrow$ espressione ben formata di tipo int
 $\Rightarrow$ assegnamento ben formato

→ blocco ben formato

Sen. dinamica

...

$\vdash \langle d; c, \emptyset \rangle \rightarrow$

↑
mem iniziale vuota